5. Technical Setup and Deployment

For developers. Full detail is in the main README; this file is the short, ordered version.

Stack

- React 19, React Router 7, Vite 8 and Tailwind CSS 3. JavaScript only.
- Framer Motion for animation.
- Supabase: Postgres, Auth, Realtime, Storage, and Edge Functions (Deno).
- Stripe Checkout, Gmail SMTP (denomailer), WhatsApp Cloud API, Instagram API with Instagram Login, and Google Places API (New).
- Hosting: Netlify for the static frontend. `public/_redirects` sends every route to `index.html`, so `/admin` and the other deep links work.

Repository layout

```
src/
  pages/            one component per page; admin/ holds the staff panel
  components/       shared UI (Header, Footer, PageHero, InstagramCard player, form/)
  hooks/            useInstagramFeed, useGoogleReviews, useDbCollection, ...
  lib/              supabaseClient, db.js (queries), api.js (Edge Function calls), stripe,
whatsapp
  data/             business.js, doctors.js, instagram.js (saved fallback posts)
public/             logo, favicons, saved Instagram images, _redirects
supabase/
  schema.sql        tables, RLS policies, functions, storage bucket. Safe to re-run.
  config.toml       per-function JWT settings
  functions/        8 Edge Functions plus _shared/
```

First-time setup, in order

1. **Install and run locally** `bash npm install cp .env.example .env # fill in the Supabase URL and anon key, and the Stripe publishable key npm run dev`
2. **Database:** in the Supabase SQL Editor, run the whole of `supabase/schema.sql`. It can be re-run safely after updates.
3. **Authentication settings** (Supabase dashboard): - **Sign In / Providers** → **Email:** turn off **Allow new users to sign up**. Staff join by invitation only. - **URL Configuration:** set **Site URL** to the live domain, and add `https://<domain>/admin/accept-invite` to **Redirect URLs**. - **Users** → **Add user:** create the clinic owner's account. This is the only account made by hand. The owner then signs in once at `/admin`; the first sign-in becomes the **super admin**, who invites all other staff from **Admin** → **Staff**.
4. **Link the CLI** `bash npm install -g supabase supabase login supabase link --project-ref <project-ref>`

5. **Secrets:** copy `supabase/functions/.env.example` to `supabase/functions/.env` (this file is git-ignored), fill it in, then run: `bash supabase secrets set --env-file supabase/functions/.env supabase secrets list # shows the names only, to check`
6. **Deploy the functions** `bash for f in create-checkout-session checkout-session-status send-booking-email \ send-contact-email stripe-webhook instagram-feed google-reviews manage-staff; do supabase functions deploy $f done`
7. **Stripe webhook:** - Endpoint: `https://<project-ref>.supabase.co/functions/v1/stripe-webhook` - Events: `checkout.session.completed`, `checkout.session.async_payment_succeeded`, `checkout.session.async_payment_failed`, `checkout.session.expired`, `charge.refunded` - Put the signing secret in `STRIPE_WEBHOOK_SECRET`.
8. **Netlify:** - Connect the GitHub repository. - Build command: `npm run build`. Publish directory: `dist`. - Add the `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY` and `VITE_STRIPE_PUBLISHABLE_KEY` environment variables. - Set `CLIENT_URL` (a Supabase secret) to the live domain.
9. **Optional: hourly Instagram sync.** This is the `pg_cron` job in README §7, so new posts appear even when nobody visits the site.

Environment variables

Netlify (built into the public bundle, so these must be public keys only): `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `VITE_STRIPE_PUBLISHABLE_KEY`

Supabase secrets (server-side only):

Group	Variables
Stripe	`STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `CLIENT_URL`, `STRIPE_PAYMENT_METHOD_TYPES`, `STRIPE_ALLOW_PROMOTION_CODES`, `STRIPE_INVOICES`
Email	`SMTP_USER`, `SMTP_PASS`, `SMTP_FROM`, `CLINIC_NOTIFY_EMAIL`
WhatsApp	`WHATSAPP_TOKEN`, `WHATSAPP_PHONE_NUMBER_ID`, `CLINIC_WHATSAPP_NUMBER`, `WHATSAPP_BOOKING_TEMPLATE`, `WHATSAPP_ORDER_TEMPLATE`, `WHATSAPP_TEMPLATE_LANG`
Instagram	`INSTAGRAM_ACCESS_TOKEN`
Google	`GOOGLE_PLACES_API_KEY`, `GOOGLE_PLACE_ID`

Any blank value switches that feature off without breaking the site. For example, with no Stripe key the shop falls back to WhatsApp orders.

Edge Functions

Function	Called by	Does
`create-checkout-session`	Cart page	Re-prices the order server-side (items + 5% VAT + delivery) and opens Stripe Checkout

Function	Called by	Does
`checkout-session-status`	Success page	Confirms the payment and marks the order paid (only once)
`stripe-webhook`	Stripe	Handles paid, failed, expired and refunded events. Called without a JWT; checks Stripe's signature instead.
`send-booking-email`	Booking page	Sends the patient and clinic emails and the WhatsApp alert. Only sends once per booking.
`send-contact-email`	Contact page	Sends contact-form enquiries
`instagram-feed`	Gallery, Home	Copies posts and videos into Storage, serves the feed, and refreshes the token weekly
`google-reviews`	Home	Live rating and reviews, cached for 6 hours
`manage-staff`	Admin → Staff	Super admin only. Lists, invites, resends links to and removes staff. Invitation and reset links are sent through the Gmail mailer and open `/admin/accept-invite`. Uses `CLIENT_URL` for the link address.

Security model

- **Row Level Security** is on for every table. Anonymous visitors can only read active products and treatments, insert bookings and orders, and call `decrement_stock()`.
- **Admin access needs a staff-list entry, not just a login.** Every admin policy checks `public.is_staff()` (a row in `public.staff`). Roles are `super_admin` and `staff`. `claim_staff_role()` makes the first signed-in user super admin while none exists, and only `manage-staff` (service role) can add or remove anyone after that.
- Only staff can read or update bookings and orders. Payment status is only ever changed by the functions, using the service-role key.
- `app_settings` and `instagram_posts` have no policies, so only the functions can access them.
- Secret keys exist only as Supabase secrets, never in the repository or in Netlify.

Releasing a change

1. Make the change on a branch and run `npm run build` locally.
2. Merge to `main`. Netlify builds and deploys automatically.
3. If a function changed, run `supabase functions deploy <name>`.
4. If `schema.sql` changed, re-run it in the SQL Editor.